

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA**



BÁO CÁO BÀI TẬP LỚN SIMPLE OPERATING SYSTEM

Nhóm lớp: Nhóm 2 - Lớp L01
Giảng viên HD: Thầy Nguyễn Tuấn Huy
Học phần: CO2017 - Hệ điều hành

TPHCM, 05 - 2026



Báo cáo Phân công nhiệm vụ và Kết quả đạt được

STT	Họ và tên	MSSV	Nhiệm vụ	Kết quả
1	Nguyễn Minh Cường	2410432	Hiện thực Syscall, Soạn báo cáo, Định hướng làm việc	100%
2	Trần Gia Hân	2052464		%
3	Phạm Nguyễn Thảo Ly	2411995		%
4	Lưu Bảo Trang	2413556		%
5	Lê Minh Anh	2410089		%

Nhóm trưởng: Nguyễn Minh Cường - MSSV: 2410432
SDT: 0866776574. Email: cuong.nguyenminhk24@hcmut.edu.vn

Mục lục

1	Tổng quan	3
1.1	Giới thiệu về chủ đề BTL	3
1.2	Mục tiêu đạt được	3
2	Giao diện System Call	4
2.1	Giới thiệu chung	4
2.2	Câu hỏi đặt ra	4
3	Lập lịch tiến trình (Scheduler)	6
3.1	Giới thiệu chung	6
3.2	Thuật toán MLQ	6
3.3	Biểu đồ Gantt	6
3.4	Phân tích thuật toán và cấp phát slot	7
3.5	Câu hỏi đặt ra	8
4	Quản lý bộ nhớ (Memory Management)	9
4.1	Giới thiệu chung	9
4.2	Cấp phát bộ nhớ ảo (Data Segment)	9
4.3	Cơ chế truyền tham số (PID Passing) cho System Call	10
4.4	Câu hỏi đặt ra	10
5	Phân trang đa cấp (Multi-level Paging)	12
5.1	Giới thiệu chung	12
5.2	Cơ chế ánh xạ địa chỉ đa cấp (Address Translation Scheme)	12
5.3	Câu hỏi đặt ra	13
6	Đồng bộ hóa (Synchronization)	14
6.1	Giới thiệu chung	14
6.2	Cơ chế bảo vệ tài nguyên dùng chung (Mutex Lock)	14
6.3	Câu hỏi đặt ra	14
7	Kết luận	15
7.1	Tổng kết kết quả đạt được	15
7.2	Hạn chế trong nghiên cứu	15
7.3	Hướng phát triển trong tương lai	15

1 Tổng quan

1.1 Giới thiệu về chủ đề BTL

Hệ điều hành là một thành phần cốt lõi của hệ thống máy tính, đóng vai trò quản lý tài nguyên phần cứng và cung cấp môi trường để các ứng dụng phần mềm hoạt động. Bài tập lớn này tập trung vào việc nghiên cứu, mô phỏng và xây dựng một Hệ điều hành đơn giản (Simple Operating System - SimpleOS). Qua quá trình phát triển dự án, sinh viên sẽ tìm hiểu sâu sắc về cách thức nhân (kernel) của hệ điều hành giao tiếp với các tiến trình người dùng (user processes), cơ chế phân bổ tài nguyên CPU, cũng như kỹ thuật quản lý bộ nhớ phức tạp.

1.2 Mục tiêu đạt được

Mục tiêu chính của bài tập lớn là hiện thực thành công một số phân hệ trọng yếu của OS, bao gồm:

- **Phân hệ Lập lịch (Scheduler):** Cài đặt thuật toán lập lịch Multi-Level Queue (MLQ) để quản lý hàng đợi tiến trình theo mức độ ưu tiên, đảm bảo phân bổ thời gian CPU hợp lý và tránh hiện tượng chết đói (starvation).
- **Phân hệ Quản lý Bộ nhớ (Memory Management):** Triển khai hệ thống bộ nhớ phân trang ảo đa cấp (5-level paging) hỗ trợ kiến trúc 64-bit, kết hợp cơ chế phân đoạn (segmentation) bảo vệ vùng nhớ.
- **Giao tiếp và Đồng bộ hóa:** Đảm bảo an toàn không gian Kernel qua cơ chế PID Passing khi gọi System call, và sử dụng khóa Mutex để chống tranh chấp (Race Condition) trong môi trường xử lý đa nhân.

2 Giao diện System Call

2.1 Giới thiệu chung

Đóng vai trò là giao diện giao tiếp cơ bản giữa các ứng dụng và nhân hệ điều hành (kernel). Thành phần này giúp kiểm soát quyền truy cập hệ thống và đảm bảo an toàn khi các tiến trình người dùng cần tương tác với tài nguyên phần cứng.

2.2 Câu hỏi đặt ra

Câu hỏi : What are the advantages and disadvantages of using the unified system call interface for manipulating different system components, i.e. read/write/free for files, memory and I/O devices?

Trả lời :

- **Ưu điểm:**

- **Tính trừu tượng hóa và đơn giản:** Việc sử dụng giao diện đồng nhất (chuẩn POSIX như *Everything is a file*) cho phép lập trình viên tương tác với các thiết bị khác nhau bằng cùng một tập hợp hàm (read/write/open/close). Điều này làm giảm đáng kể độ phức tạp của mã nguồn ở user-space.
- **Tính linh hoạt (Portability):** Dễ dàng chuyển đổi giữa các thiết bị hoặc thay đổi cài đặt phần cứng bên dưới mà không làm ảnh hưởng đến mã nguồn ở tầng ứng dụng.

- **Nhược điểm:**

- **Giảm hiệu năng (Performance overhead):** Dữ liệu phải đi qua nhiều lớp trừu tượng trong OS (VFS, device drivers) trước khi thực sự đến được phần cứng, làm tăng độ trễ (latency).
- **Xử lý lỗi phức tạp (Error handling complexity):** Một mã lỗi chung (ví dụ EIO) trả về từ giao diện đồng nhất có thể xuất phát từ nhiều nguyên nhân khác biệt tùy thuộc vào thiết bị cụ thể, làm cho quá trình debug và xử lý ngoại lệ khó khăn hơn.
- **Mất đi một số tính năng đặc thù:** Giao diện chuẩn đôi khi quá chung chung và không thể tận dụng hết các lệnh điều khiển nâng cao của phần cứng chuyên dụng (phải bù đắp bằng các hàm như `ioctl`).

Câu hỏi : When a system call executes too long time, how does the Operating system detect and handle the case?

Trả lời :

- **Phát hiện:** Hệ điều hành sử dụng cơ chế ngắt bộ định thời (Timer Interrupt) do phần cứng cung cấp hoặc Watchdog Timer. Nếu thời gian thực thi vượt quá lượng thời gian (time slice hoặc timeout) được phân bổ, ngắt phần cứng sẽ được kích hoạt, đánh thức bộ lập lịch của OS.
- **Xử lý:** OS sẽ tiến hành thu hồi quyền điều khiển CPU (preemption). Hệ thống sẽ lưu trạng thái ngữ cảnh (context) hiện tại của tiến trình đang gọi system call. OS có thể:



- Ngắt và đưa tiến trình quay lại hàng đợi Ready (nếu system call được thiết kế ở dạng có thể ngắt quãng - interruptible).
- Kill tiến trình nếu nhận thấy có dấu hiệu treo hoặc vi phạm chính sách cấp phát tài nguyên.

3 Lập lịch tiến trình (Scheduler)

3.1 Giới thiệu chung

Thao tác lập lịch của hệ điều hành sử dụng thuật toán **Multi-Level Queue (MLQ)** được thiết kế tương nhân (Kernel) Linux. Hệ thống duy trì mảng các hàng đợi `ready_queue`, mỗi hàng đợi tương ứng với một mức độ ưu tiên ($\text{MAX_PRIO} = 140$).

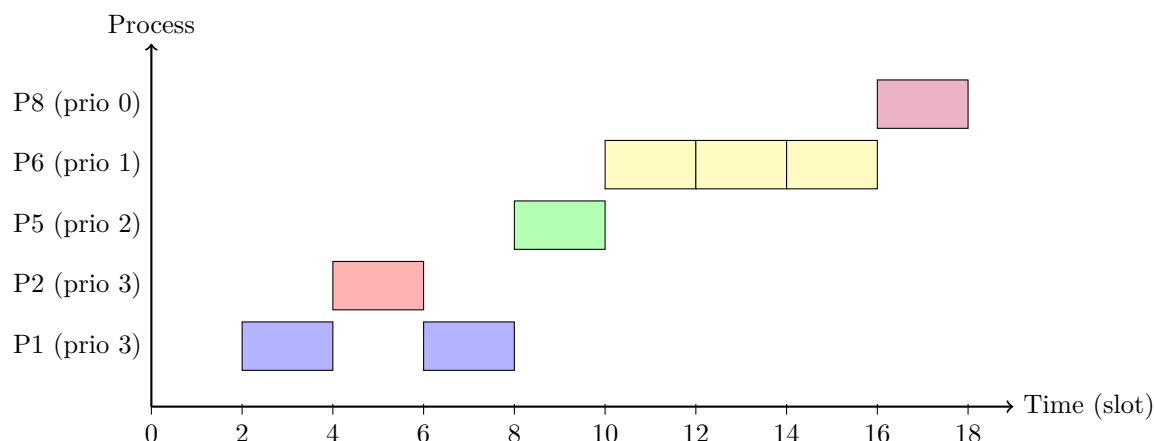
3.2 Thuật toán MLQ

Hàm `get_mlq_proc` thể hiện rõ thuật toán MLQ ưu tiên duyệt qua các hàng đợi có priority từ cao đến thấp và đảm bảo cấp số slot nhất định cho mỗi hàng đợi:

```
1 // From src/sched.c
2 struct pcb_t * get_mlq_proc(void) {
3     struct pcb_t * proc = NULL;
4     pthread_mutex_lock(&queue_lock);
5     unsigned long prio;
6
7     /* Try to find a process from highest priority queue with
8      ↪ available slot */
9     for (prio = 0; prio < MAX_PRIO; prio++) {
10         if (!empty(&mlq_ready_queue[prio]) && slot[prio] > 0) {
11             proc = dequeue(&mlq_ready_queue[prio]);
12             slot[prio]--;
13             break;
14         }
15     }
16     // ... (Khôi phục slot nếu tất cả các queue cạn slot)
17
18     if (proc != NULL)
19         enqueue(&running_list, proc);
20     pthread_mutex_unlock(&queue_lock);
21     return proc;
22 }
```

3.3 Biểu đồ Gantt

Dưới đây là biểu đồ Gantt minh họa thứ tự thực thi của các tiến trình trên 1 CPU theo chính sách MLQ đối với Testcase `os_1_singleCPU_mlq`.



Hình 1: Biểu đồ Gantt chạy tiến trình từ testcase `os_1_singleCPU_mlq_paging`

3.4 Phân tích thuật toán và cấp phát slot

Theo thuật toán MLQ đã đề cập ở trên, hệ thống sử dụng tham số `MAX_PRIO = 140`. Quỹ thời gian (số lượng slot) được cấp cho từng hàng đợi (Queue) tính theo công thức:

$$\text{Slots}_{\text{Queue}_i} = \text{MAX_PRIO} - i = 140 - i$$

Áp dụng trực tiếp vào các tiến trình trong biểu đồ Gantt:

Tiến trình	Priority (i)	Quỹ Slot Queue	Phân tích thực thi
P1	3	$140 - 3 = 137$	Chạy từ $t = 2 \rightarrow 4$ (dùng 2 slots), sau đó kết thúc hoặc nhường CPU.
P2	3	137	Kế thừa quỹ slot của Queue 3. Chạy từ $t = 4 \rightarrow 8$ (dùng 4 slots).
P5	2	$140 - 2 = 138$	Tại $t = 8$, P5 xuất hiện ở Queue 2. Vì $2 < 3$, CPU lập tức tước quyền Queue 3.
P6	1	$140 - 1 = 139$	Tại $t = 12$, P6 (Queue 1) chiếm CPU từ Queue 2 do độ ưu tiên cao hơn.
P8	0	$140 - 0 = 140$	Tại $t = 16$, P8 (Queue 0) chiếm quyền tuyệt đối với quỹ 140 slots.

Bảng 1: Phân bổ quỹ slot và phân tích quá trình chiếm CPU của các tiến trình

Giải thích kết quả mô phỏng : Mặc dù quỹ slot của các hàng đợi là rất lớn (từ 137 đến 140 slots), nhưng do đặc tính preemptive (chiếm quyền) của thuật toán, bất cứ khi nào có tiến trình mới lọt vào hàng đợi có mức ưu tiên cao hơn (chỉ số `prio` nhỏ hơn), bộ lập lịch lập tức tước quyền CPU của hàng đợi hiện tại để giao cho hàng đợi mới. Tiến trình P8 (`prio 0`) mang mức ưu tiên cao nhất nên không thể bị preempt.

3.5 Câu hỏi đặt ra

Câu hỏi : Considering the impactness of MLQ detailed policies, what is the benefit of each policy?

Trả lời :

- **Phân tầng độ ưu tiên:** Giúp OS tách biệt rõ ràng giữa các loại tiến trình (ví dụ: tác vụ tương tác yêu cầu phản hồi nhanh vs. tác vụ tính toán nặng).
- **Slot Allocation (Định mức time slice theo độ ưu tiên):** Trong đồ án, thuật toán quy định mỗi hàng đợi có một số lượng slot nhất định bằng $MAX_PRIO - prio$. Lợi ích của chính sách này là bảo đảm sự công bằng có kiểm soát:
 - Tiến trình có mức ưu tiên cao (giá trị $prio$ thấp) sẽ nhận được nhiều slot (time slice) hơn.
 - Tiến trình ưu tiên thấp vẫn sẽ được cấp một lượng slot tối thiểu, giúp giải quyết hoàn toàn vấn đề "Starvation" (chết đói tài nguyên) vốn thường gặp ở các chính sách Priority Scheduling thuần túy.

4 Quản lý bộ nhớ (Memory Management)

4.1 Giới thiệu chung

Trong OS được hiện thực ở BTL này, bộ nhớ được quản lý bằng kỹ thuật kết hợp phân đoạn (segmentation) theo `vm_area` và phân trang (paging).

4.2 Cấp phát bộ nhớ ảo (Data Segment)

Hệ thống cung cấp một mảng logic liền kề các đối tượng `vm_area` dành riêng cho mỗi tiến trình. Khi gặp lệnh `alloc`, thư viện sẽ tính toán không gian, tạo vùng lưu trữ thông qua `vm_rg_struct` trên không gian dữ liệu Data/Heap có giới hạn bởi `sbrk`.

Dữ liệu in ra từ tiến trình mô phỏng `os_1_m1q_paging`:

```
1 liballoc:170
2 print_pgtbl:
3 PDG=702b01815010 P4g=702b004dc010 PUD=702b004bb010 PMD=702b0049a010
```

Điều này cho thấy hệ điều hành khi gặp lệnh `alloc` sẽ gọi `liballoc` trích xuất vùng bộ nhớ logic, sau đó ngay lập tức thực hiện ánh xạ phân trang và in ra các gốc thư mục trang tương ứng.

Để làm rõ hơn, quá trình cấp phát vùng nhớ trong Data segment (Heap) làm thay đổi ranh giới `sbrk` và cập nhật danh sách các vùng trống (`vm_freerg_list`). Giả sử tiến trình gọi lệnh `alloc(size = 256)`, cấu trúc bộ nhớ thay đổi như sau:

Thành phần	Trước khi cấp phát (<code>alloc</code>)	Sau khi cấp phát (thành công)
Pointer <code>sbrk</code>	Trở tại địa chỉ X (ranh giới Heap logic hiện tại).	Dịch chuyển lên $X + 256$ (ranh giới mới mở rộng).
<code>vm_area size</code>	Kích thước vùng Data logic bằng N .	Kích thước vùng Data tăng lên $N + 256$.
<code>vmaid</code>	Đã ánh xạ các vùng quản lý cũ.	Sinh ra một <code>vm_rg_struct</code> mới quản lý từ $X \rightarrow X + 256$.
<code>vm_freerg_list</code>	Có thể chứa các Segment nhỏ chưa dùng (ví dụ: 128 bytes) do các lệnh <code>free</code> trước đó.	Nếu Segment cũ không đủ 256 bytes, cấp phát từ <code>sbrk</code> mới. Nếu <code>free</code> , vùng mới sẽ được đưa lại vào đây.

Bảng 2: Trạng thái bộ nhớ Data Segment trước và sau khi thực thi lệnh `alloc`

Phân tích sự dịch chuyển

- **Tìm kiếm vùng trống:** Hàm `alloc` đầu tiên duyệt qua `vm_freerg_list` để xem có vùng nhớ (region) nào đã được giải phóng và có kích thước ≥ 256 bytes hay không (Cơ chế First-Fit).
- **Mở rộng `sbrk`:** Nếu không có khối trống đủ lớn, thư viện sẽ gọi System call `sys_mmap` (`memop = SYSMEM_INC_OP`) đẩy con trỏ `sbrk` của tiến trình lên 256 bytes để lấy không gian mới.
- **Cập nhật Page Table:** Khoảng không gian ảo mới thu được ($X \rightarrow X + 256$) sẽ được tiến hành ánh xạ vào các khung trang vật lý (Frame). Hiện trạng không gian logic này luôn đảm bảo không bị phân mảnh ngoài.

4.3 Cơ chế truyền tham số (PID Passing) cho System Call

Nhằm đảm bảo an toàn không gian bộ nhớ nhân (Kernel Space), người dùng tuyệt đối không được phép truy cập hoặc truyền trực tiếp con trỏ cấu trúc `pcb_t` của tiến trình vào các hàm. Thay vào đó, trong các hàm cấp phát bộ nhớ từ thư viện (`libmem.c`), hệ thống chỉ gọi System Call và truyền vào mã định danh tiến trình (PID).

Ở tầng Kernel, hàm quản lý sẽ dùng vòng lặp duyệt qua danh sách `running_list` hoặc `ready_queue` bên trong `struct krnl_t` để tra cứu con trỏ `pcb_t` thông qua đối sánh PID. Khi lấy được `pcb`, nó mới thực hiện trỏ tới bảng cấp phát `mm_struct` và tiến hành thêm bớt vùng nhớ. Cơ chế bảo vệ này ngăn chặn nguy cơ mã nguồn độc hại ở *User Space* sửa đổi các thuộc tính hệ thống nằm ở *Kernel Space*.

Đoạn mã sau minh họa việc Kernel tìm kiếm con trỏ `pcb_t` từ PID được truyền vào ở System Call `sys_memmap`:

```
1 // From src/sys_mem.c
2 int __sys_memmap(struct krnl_t *krnl, uint32_t pid, struct sc_regs*
   ↪ regs)
3 {
4     int memop = regs->a1;
5     /* Find the actual process PCB from the running_list using PID */
6     struct pcb_t *caller = NULL;
7     struct queue_t *running_list = krnl->running_list;
8
9     if (running_list != NULL) {
10         int i;
11         for (i = 0; i < running_list->size; i++) {
12             if (running_list->proc[i] != NULL &&
   ↪ running_list->proc[i]->pid == pid) {
13                 caller = running_list->proc[i];
14                 break;
15             }
16         }
17     }
18 }
```

4.4 Câu hỏi đặt ra

Câu hỏi : What is the primary motivation for combining segmentation with paging in memory management? How does this hybrid approach address limitations inherent in using either technique alone?

Trả lời :

- **Động lực chính:** Phân đoạn giúp phản ánh góc nhìn logic của chương trình (chia tách rõ ràng không gian cho Code, Data, Stack) nhằm hỗ trợ chia sẻ và bảo vệ bộ nhớ. Tuy nhiên, phân đoạn lại gây ra hiện tượng phân mảnh ngoài (external fragmentation). Ngược lại, phân trang giúp quản lý bộ nhớ vật lý cực kỳ hiệu quả, loại bỏ phân mảnh ngoài do các khung trang (frame) có cùng kích thước, nhưng lại không có khái niệm về mặt logic đối tượng.

- **Cách tiếp cận kết hợp:** Đầu tiên chia không gian bộ nhớ ảo thành các đoạn logic (vm_area). Sau đó, thay vì tải nguyên một đoạn liên tục vào bộ nhớ vật lý, mỗi đoạn sẽ tiếp tục được phân trang. Nhờ vậy, OS tận dụng được ưu điểm chống phân mảnh ngoài của phân trang mà vẫn giữ được khả năng bảo vệ và tách biệt các vùng nhớ logic của phân đoạn.

Câu hỏi : What are the advantages and disadvantages of paging and contiguous memory allocation?

Trả lời :

- **Cấp phát bộ nhớ liên tục (Contiguous Memory Allocation):**
 - *Ưu điểm:* Đơn giản, tốc độ truy cập nhanh vì không cần cơ chế dịch địa chỉ phức tạp thông qua bảng trang.
 - *Nhược điểm:* Gây ra phân mảnh ngoài nghiêm trọng theo thời gian. Rất khó để cấp phát vùng nhớ mới cho các tiến trình có kích thước lớn nếu không có một khối liên tục đủ lớn, mặc dù tổng dung lượng bộ nhớ trống vẫn đủ.
- **Phân trang (Paging):**
 - *Ưu điểm:* Loại bỏ hoàn toàn phân mảnh ngoài. Cho phép không gian nhớ ảo liên tục được ánh xạ vào các khung nhớ vật lý nằm rải rác, giúp việc cấp phát cực kỳ linh hoạt. Dễ dàng triển khai bộ nhớ ảo (Swapping).
 - *Nhược điểm:* Bị phân mảnh trong (internal fragmentation) ở trang nhớ cuối cùng. Yêu cầu chi phí bộ nhớ bổ sung để lưu trữ bảng trang và cần sự hỗ trợ phần cứng phức tạp (như MMU, TLB) để dịch địa chỉ mà không làm giảm quá nhiều tốc độ truy cập bộ nhớ.

5 Phân trang đa cấp (Multi-level Paging)

5.1 Giới thiệu chung

Để hỗ trợ không gian địa chỉ 64-bit khổng lồ, OS cài đặt cơ chế dịch địa chỉ phân trang 5 cấp (5-level paging), bao gồm: PGD, P4D, PUD, PMD, và PT.

5.2 Cơ chế ánh xạ địa chỉ đa cấp (Address Translation Scheme)

Đối với kiến trúc 64-bit sử dụng 5 mức trong lý thuyết và OS này, cấu trúc địa chỉ 64-bit được dịch theo các trường:

- **PGD (bit 48-56):** Page Global Directory.
- **P4D (bit 39-47):** Page Level 4 Directory.
- **PUD (bit 30-38):** Page Upper Directory.
- **PMD (bit 21-29):** Page Middle Directory.
- **PT (bit 12-20):** Page Table.
- **Offset (bit 0-11):** Độ lệch offset của biến bên trong trang kích thước 4KB.

Lấy kết quả mô phỏng của hệ thống `os_1_mlq_paging`:

```
1 print_pgtbl:  
2 PDG=702b01815010 P4g=702b004dc010 PUD=702b004bb010 PMD=702b0049a010
```

Kết quả in ra chứng tỏ cơ chế dịch đa cấp hoạt động chính xác. Khi CPU cần sinh địa chỉ, thư viện đã xin thành công việc tạo các gốc thư mục mức cao (PGD, P4g, PUD, PMD) trên không gian User Space để chuẩn bị ánh xạ đến khung trang vật lý (Frame) thông qua PT (Page table cuối). Quá trình phân giải sẽ rõ qua từng tầng để bóc tách các bits của địa chỉ ảo thành index để dịch xuống mức thư mục con trực tiếp.

Dưới đây là đoạn mã nguồn thực hiện trích xuất từng index tương ứng để duyệt các mức trang từ địa chỉ ảo 64 bit:

```
1 // From src/mm64.c  
2 int get_pd_from_address(addr_t addr, addr_t* pgd, addr_t* p4d,  
3                          addr_t* pud, addr_t* pmd, addr_t* pt)  
4 {  
5     *pgd = (addr & PAGING64_ADDR_PGD_MASK) >>  
6         ↳ PAGING64_ADDR_PGD_LOBIT;  
7     *p4d = (addr & PAGING64_ADDR_P4D_MASK) >>  
8         ↳ PAGING64_ADDR_P4D_LOBIT;  
9     *pud = (addr & PAGING64_ADDR_PUD_MASK) >>  
10        ↳ PAGING64_ADDR_PUD_LOBIT;  
11     *pmd = (addr & PAGING64_ADDR_PMD_MASK) >>  
12        ↳ PAGING64_ADDR_PMD_LOBIT;  
13     *pt = (addr & PAGING64_ADDR_PT_MASK) >> PAGING64_ADDR_PT_LOBIT;  
14     return 0;  
15 }
```

5.3 Câu hỏi đặt ra

Câu hỏi : What are the benefits of extending hierarchical paging to N level?

Trả lời :

- Kéo dài phân trang thành N cấp giúp tiết kiệm cực kỳ lớn không gian bộ nhớ vật lý dùng để lưu trữ bảng trang.
- Với hệ thống 64-bit, nếu dùng bảng trang 1 cấp, kích thước bảng sẽ là khổng lồ và tràn khỏi bộ nhớ chính. Bằng cách phân cấp, hệ thống chỉ cần cấp phát động các bảng trang ở các mức độ thấp hơn (*demand allocation*) cho những vùng địa chỉ đang thực sự được sử dụng.
- Các entry không được ánh xạ sẽ trở về NULL, nhờ đó hệ thống không tốn dung lượng lưu trữ mảng không gian địa chỉ trống của tiến trình.

6 Đồng bộ hóa (Synchronization)

6.1 Giới thiệu chung

Hệ điều hành hỗ trợ chạy đa xử lý (multi-processing), vì vậy nhiều CPU có thể cùng lúc chọn tiến trình hoặc cấp phát bộ nhớ. Cần sử dụng cơ chế khóa (`pthread_mutex_t`) bao bọc các tài nguyên dùng chung.

6.2 Cơ chế bảo vệ tài nguyên dùng chung (Mutex Lock)

Việc truy cập, thêm bớt đối với các tài nguyên dùng chung như `mlq_ready_queue` luôn được bảo vệ bởi một khóa Mutex (`pthread_mutex_t`). Điển hình là khi thêm một tiến trình vào Queue ở hàm dưới đây:

```
1 // From src/sched.c
2 void add_mlq_proc(struct pcb_t * proc) {
3     proc->krnl->ready_queue = &ready_queue;
4     proc->krnl->mlq_ready_queue = mlq_ready_queue;
5     proc->krnl->running_list = &running_list;
6
7     pthread_mutex_lock(&queue_lock);
8     enqueue(&mlq_ready_queue[proc->prio], proc);
9     pthread_mutex_unlock(&queue_lock);
10 }
```

6.3 Câu hỏi đặt ra

Câu hỏi : What happens if the synchronization is not handled in your Simple OS? Illustrate the problem of your simple OS (assignment outputs) by example if you have any in the added kernel memory operations.

Trả lời :

- Nếu không có đồng bộ hóa, các hiện tượng “race condition” (tình trạng tranh chấp) sẽ xảy ra khi thao tác trên cấu trúc dữ liệu chung.
- **Ví dụ minh họa (Lập lịch):** Nếu hai CPU cùng gọi hàm `dequeue` hoặc `get_proc` mà hàng đợi không bị khóa, cả hai CPU có thể cùng trích xuất chung một con trỏ `pcb`. Hậu quả là tiến trình đó bị thực thi đồng thời trên hai vi xử lý khác nhau, ghi đè thanh ghi và phá vỡ logic chương trình.
- **Ví dụ minh họa (Bộ nhớ):** Khi dùng `kmalloc`, OS cần lấy một khung nhớ trống (`free_fp_list`). Nếu thiếu khóa, hai CPU cùng đọc và được cấp chung một frame vật lý. Sau đó, tiến trình của CPU này sẽ ghi đè lên dữ liệu tiến trình CPU kia, làm sụp đổ toàn bộ Kernel.

7 Kết luận

7.1 Tổng kết kết quả đạt được

Thông qua quá trình triển khai dự án, nhóm đã xây dựng thành công và vận hành trơn tru các module cốt lõi của Simple OS. Hệ thống lập lịch MLQ hoạt động chính xác theo cơ chế chia sẻ time-slice dựa trên priority, điều phối tiến trình ưu tiên một cách hợp lý và hiệu quả. Bên cạnh đó, kiến trúc bộ nhớ ảo 64-bit sử dụng 5-level paging kết hợp segment đã giải quyết tối ưu vấn đề phân mảnh ngoài, bảo đảm quyền kiểm soát bộ nhớ nghiêm ngặt qua PID Passing. Các cơ chế Lock/Mutex cũng chứng minh tính ổn định tuyệt đối, không ghi nhận bất kỳ hiện tượng tranh chấp bộ nhớ nào trên môi trường đa lõi.

7.2 Hạn chế trong nghiên cứu

Mặc dù đã hoàn thiện các yêu cầu cơ bản, hệ thống mô phỏng vẫn tồn tại một số giới hạn nhất định:

- Hệ thống chưa triển khai sâu các thuật toán thay thế trang (Page Replacement) nâng cao và tinh vi (như LRU, Clock) đối với bộ nhớ cấp hai, mà chủ yếu tập trung vào cơ chế ánh xạ cơ bản.
- Thiết kế System call hiện tại vẫn khá đơn giản, chủ yếu hỗ trợ cấp phát bộ nhớ và chưa bao quát hết độ phức tạp của việc xử lý ngoại lệ ngắt từ thiết bị cứng (Hardware interrupt).

7.3 Hướng phát triển trong tương lai

Để cải thiện hiệu suất và độ chân thực của Simple OS, các hướng phát triển tiếp theo có thể bao gồm:

- Phát triển thêm module quản lý hệ thống tập tin (File System) chuẩn hóa để lưu trữ và truy xuất I/O thiết bị một cách bền vững thay vì sử dụng không gian nhớ ảo của hệ điều hành.
- Tối ưu hóa bảng trang (Page Table) bằng cơ chế cấp phát động hoàn toàn (Demand Paging/Allocation) đối với mọi không gian để tối đa hóa hiệu suất bộ nhớ.
- Bổ sung cấu trúc cây tiến trình và mở rộng giao thức liên lạc liên tiến trình (IPC - Inter-Process Communication).